Name - Anwesa Mondal

Enrollment no. - 0400117 2024

# DAA Assignment - 2

1) Explain DFS and BFS graph algorithms.

Ans) BFS (Breadth First Search) → It explores all neighbours of node before going to the next level.

a) Start from a source node s.
b) Mark s as visited
c) Insert s into a queue.
d) While queue is not empty:
   i) Remove front node u.
   ii) Visit u (print/process it)
   iii) For every adjacent node v of u:
      If v is not visited:
      → Mark v as visited
      → Insert v into queue.

Time complexity = $O(V+E)$

Use cases - Generally used for finding shortest path in a graph, social media connections, web crawling and network broadcasting.

DFS (Depth first Search) - Explores graph by going as deep as possible in a branch.

a) Start from source node s.

b) Mark s as visited.

c) Push s into stack.

d) While stack is not empty:

i) Pop top node u.

ii) Mark u as visited

iii) For every adjacent node v of u:

If v is not visited:

→ Mark v as visited

→ Push v into stack.

Time complexity = $O(V+E)$

Use cases - Cycle detection in graphs and topological sort.

2) Explain Prim's and Kruskal's algorithm for MST.

Ans) Prim's algorithm → Builds MST by starting from a node and adding minimum edge weights.

Prim (G, w, r) {

1) For each vertex $u \in G.V$

key[u] = ∞

π[u] = NIL

2) key[r] = 0

3) Q = G.V // priority queue (min heap) based on key[].

4) While Q ≠ ∅

u = Extract - min(Q)

For each vertex $v \in adj[u]$:

If $v \in Q$ and $w(u,v) <$ key[v]

π[v] = u

key[v] = w(u, v)

5) MST = { (v, π[v]} for all v ∈ G V - {r} }

6) return MST

}

Kruskal's algorithm - Builds MST by selecting smallest edges without cycle.

Kruskal (G) {

1) MST = ∅

2) Sort all edges of G in non decreasing order by weight for each vertex v in G :

Make_set (v)

3) For each edge (u, v) in sorted edges:

If Find_set (u) ≠ Find_set (v):

MST = MST ∪ {(u, v)}

UNION (u, v)

4) return MST

}

→ Make_set (v): each node starts as its own set.

→ Find_set (v): Tells which component v belongs to.

→ UNION (u, v): merges 2 components

Time complexity = O (E log E)

3) Explain Bellman Ford algorithm.

Ans) Finds shortest path even with negative weights.

Bellman Ford (G, W, S) {

1) For each vertex v ∈ V[G]:

{ d [v] = ∞

parent [v] = NIL

}

2) d [s] = 0

3) For i = 1 to |V[G]| - 1 :

For each edge (u, v) ∈ E[G] :

If d[v] > d [u] + w (u, v) :

d[v] = d[u] + w(u, v)

parent [v] = u

4) For each edge $(u, v) \in E[G]$:

If $d[v] > d[u] + w(u, v)$:

return false // cycle present with negative weights.

print (d)

5) return true.

Time complexity = $O(V \cdot E)$

4) Explain NP complete problem.

Ans) A problem is NP complete if:

i) It is in NP. Given a proposed solution, you can verify whether it is correct in polynomial time.

ii) It is NP hard. Every other problem in NP can be reduced to this problem in polynomial time. This means it is as hard as every problem in NP.

5) Explain string matching with finite automata.

Ans) String matching with finite automata is a technique to find all occurrences of pattern P in a text T by building a state machine that processes the text character by character.

→ A finite automation is defined by:

Q → set of states

E → Input character

S → Transition function

$q_0$ → start state

F/A → Final / accepting states

Transition function → $S(q, a)$

q → current state

a → input character

S tells where to move next.

Finite automation (T, s, m):

1) n = length of T

2) q = 0 // Start state

3) For i=1 to n:

    q = S(q, T[i]) // transition

    if q==m;

        print ("Pattern found ending at index", i)

Start from state 0 (no match). For each character in text, move to next state using S. If state becomes m, pattern is found. Starting index = i-m+1.

b) Explain NP completeness and reducibility.

Ans) 1) X is said to belong to NP if its solution can be verified in polynomial time.

2) X is NP hard → every problem in NP is polynomial time reducible to X.

NP complete problems are the hardest problems within NP. Solving one efficiently means solving all of NP efficiently.

Reducibility is the core tool used to prove NP completeness.

Problem A is polynomial time reducible to problem B, written as $A \leq_p B$, if there exists a polynomial time algorithm that transforms any instance of A into an instance of B such that the answer is preserved (yes → yes, no → no).

If you can solve B, you can solve A, i.e., B is at least as hard as A. This the intuition behind the above concept.